Functional Specification Document

Enhanced Leveraged ETF Strategy v2.0 - Automated Trading System

Objective

Develop a Python program that:

- Ingests historical QQQ price and volume data from an Excel file.
- Implements the strategy rules to automatically trigger buy/sell/hold actions for TQQQ and SQQQ.
- Produces a complete trade ledger book recording all simulated transactions and daily positions.
- Outputs performance metrics and equity curves for analysis.

1. Input Specifications

1.1 Data File Requirements

- **Accepted Format:** Excel (.xlsx) or CSV (.csv)
- **Required Columns:**
  - `Date` (format: YYYY-MM-DD)
  - `QQQ_Close` (closing price)
  - `QQQ_High` (daily high)
  - `QQQ_Low` (daily low)
  - `QQQ_Volume` (shares traded)
  - `TQQQ_Close` (closing price for execution)
  - `SQQQ_Close` (closing price for execution)

1.2 Configuration Parameters (User-Configurable)

```
config = {
    'initial_capital': 10000,           # Starting portfolio value
    'transaction_fee': 0.00,            # Per-trade fee ($ or %)
    'slippage': 0.001,                  # 0.1% slippage assumption
    'sma_slope_lookback': 5,            # Days for slope calculation
    'buffer_entry_50sma': 0.015,        # 1.5% above 50-SMA for entry
```

```
    'buffer_exit_50sma': 0.025,         # 2.5% below 50-SMA for exit
    'buffer_entry_200sma': 0.02,        # 2% above 200-SMA
    'buffer_exit_200sma': 0.03,         # 3% below 200-SMA
    'stop_loss_100_tqqq': 0.08,         # 8% hard stop for 100% TQQQ
    'stop_loss_50_tqqq': 0.06,          # 6% hard stop for 50% TQQQ
    'trailing_stop_trigger': 0.15,      # 15% gain to activate trailing
    'trailing_stop_distance': 0.10,     # 10% trailing stop
    'atr_threshold_normal': 0.025,      # 2.5% in normal markets
    'atr_threshold_bull': 0.030,        # 3.0% in strong trends
    'atr_threshold_choppy': 0.020,      # 2.0% in transitions
    'atr_threshold_crisis': 0.015,      # 1.5% in crisis
}
```

2. Technical Indicator Calculations

2.1 Simple Moving Averages (SMA)

```
# Calculate for each day t:
SMA_50[t] = mean(QQQ_Close[t-49:t+1])   # 50-day simple moving average
SMA_200[t] = mean(QQQ_Close[t-199:t+1]) # 200-day simple moving average
```

2.2 SMA Slope Calculation (CRITICAL FORMULA)

```
# Calculate percentage slope over last N days (default N=5):
# Formula: ((SMA_today - SMA_N_days_ago) / SMA_N_days_ago) / N * 100

def calculate_sma_slope(sma_series, lookback=5):
    """
    Calculate annualized slope percentage for SMA

    Args:
        sma_series: pandas Series of SMA values
        lookback: number of days to calculate slope (default 5)

    Returns:
        Slope as percentage per week
    """
    sma_today = sma_series.iloc[-1]
    sma_n_days_ago = sma_series.iloc[-lookback-1]

    # Calculate percentage change per day, then annualize to per week
    slope_pct_per_day = ((sma_today - sma_n_days_ago) / sma_n_days_ago) / lookback * 100
    slope_pct_per_week = slope_pct_per_day * 5  # Convert to weekly rate

    return slope_pct_per_week

# Apply to both SMAs:
Slope_50[t] = calculate_sma_slope(SMA_50[t-lookback:t+1], lookback=5)
Slope_200[t] = calculate_sma_slope(SMA_200[t-lookback:t+1], lookback=5)
```

Example Calculation:

- Day 0: SMA_50 = 615.00
- Day 5: SMA_50 = 606.88
- Slope = ((606.88 - 615.00) / 615.00) / 5 × 100 = -0.264% per day
- Weekly slope = -0.264% × 5 = **-1.32% per week**

2.3 Combined Momentum Score

```
# Weighted blend of both slopes:
Combined_Momentum[t] = (Slope_50[t] * 0.6) + (Slope_200[t] * 0.4)

# Example:
# Slope_50 = -1.32% per week
# Slope_200 = +0.85% per week
# Combined = (-1.32 * 0.6) + (0.85 * 0.4) = -0.792 + 0.34 = -0.452% per week
```

2.4 Average True Range (ATR)

```
# Calculate True Range for each day:
TR[t] = max(
    QQQ_High[t] - QQQ_Low[t],
    abs(QQQ_High[t] - QQQ_Close[t-1]),
    abs(QQQ_Low[t] - QQQ_Close[t-1])
)

# 14-day ATR:
ATR_14[t] = mean(TR[t-13:t+1])

# ATR as percentage of price:
ATR_Percent[t] = (ATR_14[t] / QQQ_Close[t]) * 100
```

2.5 Volume Analysis

```
# 20-day average volume:
Avg_Volume_20[t] = mean(QQQ_Volume[t-19:t+1])

# Current volume ratio:
Volume_Ratio[t] = QQQ_Volume[t] / Avg_Volume_20[t]

# Volume confirmation multiplier:
if Volume_Ratio[t] >= 1.25:
    Volume_Multiplier[t] = 1.00      # Full size (strong confirmation)
elif Volume_Ratio[t] >= 1.10:
    Volume_Multiplier[t] = 0.75      # 75% size (decent confirmation)
elif Volume_Ratio[t] >= 0.90:
    Volume_Multiplier[t] = 0.50      # 50% size (average volume)
else:
    Volume_Multiplier[t] = 0.25      # 25% size (weak confirmation)
```

2.6 Weekly Timeframe Alignment (Optional Enhancement)

```
# Resample to weekly data (Friday close or last available):
Weekly_SMA_10 = mean(QQQ_Close_Weekly[t-9:t+1])
Weekly_Slope = calculate_sma_slope(Weekly_SMA_10, lookback=2)

# Alignment check:
Multi_Timeframe_Aligned[t] = (
    (Slope_50[t] > 0 and Weekly_Slope > 0) or
    (Slope_50[t] < 0 and Weekly_Slope < 0)
)
```

3. Trade Decision Engine (Rule-Based Logic)

3.1 Market Regime Detection

```
def determine_market_regime(price, sma_50, sma_200, slope_50, slope_200):
    """
    Determine current market regime for ATR threshold adjustment

    Returns: 'bull', 'bear', 'choppy', 'crisis'
    """
    if price > sma_200 * 1.10 and slope_200 > 0.3:
        return 'bull'           # Strong uptrend
    elif price < sma_200 * 0.90 and slope_200 < -0.3:
        return 'crisis'         # Severe downtrend
    elif abs(slope_50) < 0.1 and abs(slope_200) < 0.1:
        return 'choppy'         # Sideways/transition
    else:
        return 'normal'         # Default
```

3.2 Priority 1: Override Rules (Evaluated First)

Rule 1: Death Cross Detection

```
def check_death_cross(sma_50, sma_200):
    """
    Immediate exit signal - highest priority
    Returns: True if death cross occurs
    """
    return sma_50 < sma_200

# Action: EXIT ALL POSITIONS → 100% CASH
```

Rule 2: High Volatility Override

```
def check_high_volatility(atr_percent, regime):
    """
    Block new entries during high volatility
    """
    thresholds = {
        'bull': 0.030,
        'normal': 0.025,
        'choppy': 0.020,
        'crisis': 0.015
    }
    return atr_percent > thresholds.get(regime, 0.025)

# Action: HOLD current position or MOVE TO CASH (no new entries)
```

3.3 Priority 2: Position Entry Rules

Rule 3: 100% TQQQ Entry

```
def trigger_100_tqqq(price, sma_50, sma_200, combined_momentum,
                     volume_ratio, regime, atr_high_vol):
    """
    Full leveraged long position
    """
    conditions = [
        price > sma_50 * (1 + 0.015),           # Above 50-SMA + 1.5% buffer
        price > sma_200,                         # Above 200-SMA
        combined_momentum > 0.5,                 # Strong positive momentum
        volume_ratio >= 1.10,                    # Volume confirmation
        not atr_high_vol,                        # Volatility acceptable
        regime in ['bull', 'normal']             # Favorable regime
    ]
    return all(conditions)

# Action: BUY TQQQ to 100% of capital
# Set stop loss at entry_price * (1 - 0.08)  # 8% hard stop
```

Rule 4: 75% TQQQ + 25% Cash

```
def trigger_75_tqqq(price, sma_50, sma_200, combined_momentum, volume_ratio):
    """
    Moderate leveraged position
    """
    conditions = [
        price > sma_50 * (1 + 0.015),
        price > sma_200,
        0.2 <= combined_momentum <= 0.5,         # Moderate momentum
        volume_ratio >= 0.90,
    ]
    return all(conditions)
```

```
# Action: BUY TQQQ to 75% of capital, hold 25% cash
# Set stop loss at entry_price * (1 - 0.06)  # 6% hard stop
```

Rule 5: 50% TQQQ + 50% Cash

```
def trigger_50_tqqq(price, sma_50, sma_200, combined_momentum):
    """
    Conservative leveraged position
    """
    conditions = [
        price > sma_200,                        # Above long-term support
        -0.2 <= combined_momentum <= 0.2,       # Weak/neutral momentum
        price > sma_50 * 0.98,                  # Near or above 50-SMA
    ]
    return all(conditions)

# Action: BUY TQQQ to 50% of capital, hold 50% cash
# Set stop loss at entry_price * (1 - 0.06)  # 6% hard stop
```

Rule 6: 100% CASH

```
def trigger_cash(price, sma_50, sma_200, combined_momentum):
    """
    Defensive positioning - default safe state
    """
    conditions = [
        sma_50 < price < sma_200,               # Between SMAs
        # OR
        abs(combined_momentum) < 0.2,           # Weak momentum
        # OR
        price < sma_50 and price > sma_200      # Below 50, above 200
    ]
    return any(conditions)

# Action: SELL all positions → 100% CASH
```

Rule 7: 50% SQQQ + 50% Cash

```
def trigger_50_sqqq(price, sma_50, sma_200, combined_momentum, volume_ratio):
    """
    Moderate inverse position (bearish)
    """
    conditions = [
        price < sma_50 * (1 - 0.025),           # Below 50-SMA - 2.5% buffer
        price > sma_200,                        # Still above 200-SMA
        -0.5 <= combined_momentum < -0.2,       # Moderate negative momentum
        volume_ratio >= 1.10,                   # Volume confirmation
    ]
    return all(conditions)
```

```
# Action: BUY SQQQ to 50% of capital, hold 50% cash
# Set stop loss at entry_price * (1 + 0.06)  # 6% stop (inverse direction)
```

Rule 8: 100% SQQQ

```
def trigger_100_sqqq(price, sma_50, sma_200, combined_momentum,
                     volume_ratio, death_cross):
    """
    Full inverse position (strongly bearish)
    """
    conditions = [
        price < sma_50 * (1 - 0.025),            # Below 50-SMA - buffer
        price < sma_200 * (1 - 0.03),            # Below 200-SMA - buffer
        combined_momentum < -0.5,                # Strong negative momentum
        volume_ratio >= 1.25,                    # Strong volume confirmation
        death_cross,                             # Death cross occurred
    ]
    return all(conditions)

# Action: BUY SQQQ to 100% of capital
# Set stop loss at entry_price * (1 + 0.08)  # 8% stop (inverse)
```

3.4 Priority 3: Exit Rules

Rule 9: Exit TQQQ Positions

```
def trigger_exit_tqqq(price, sma_50, entry_price, stop_loss_price,
                      combined_momentum, trailing_stop_price=None):
    """
    Exit long leveraged positions
    """
    exit_conditions = [
        price < sma_50 * (1 - 0.03),             # Below 50-SMA - 3% buffer
        combined_momentum < -0.2,                # Momentum turned negative
        price <= stop_loss_price,                # Hard stop hit
        trailing_stop_price and price <= trailing_stop_price,  # Trailing stop
    ]
    return any
```